PYTHON PROGRAMMING

CASE STUDY ASSIGNMENT – 7

Case Study 7 – Multi-Product Shopping Budget Constraint

1. Problem Statement

Managing a shopping budget can be challenging when purchasing multiple products with different prices and quantities. Customers need to calculate the total cost of their purchases and check whether the total amount stays within their available budget.

The Multi-Product Shopping Budget Constraint system is a Python-based program that accepts the customer's budget and details of multiple products, including their names, prices, and quantities. It calculates the total cost of the shopping list and determines whether the purchase is within the budget or exceeds it. This system helps users manage their spending and make better purchasing decisions.

2. Objectives

1. **To accept the customer's shopping budget.**
2. **To collect the names, prices, and quantities of multiple products.**
3. **To calculate the total cost of each product.**
4. **To calculate the overall shopping cost.**
5. **To compare the total shopping cost with the available budget.**
6. **To display whether the shopping is within budget or exceeds the budget.**
7. **To validate product prices, quantities, and budget values.**
8. **To apply Python concepts such as loops, conditional statements, lists, and arithmetic operations to solve a real-world problem.**

3. Assumptions

1. **The customer must enter a positive shopping budget.**
2. **The customer must enter at least one product.**
3. **Each product has a name, price per unit, and quantity.**

4. **Product prices must be positive numbers.**
5. **Product quantities must be positive integers.**
6. **The cost of each product is calculated by multiplying its price by its quantity.**
7. **The total shopping cost is calculated by adding the costs of all products.**
8. **If the total cost is less than or equal to the budget, the purchase is considered within budget.**
9. **If the total cost exceeds the budget, the program displays the extra amount required.**
10. **Discounts, taxes, and delivery charges are not included in the calculations.**

4. Algorithm

1. **Start the program.**
2. **Input the customer's shopping budget.**
3. **If the budget is less than or equal to zero, display an invalid budget message.**
4. **Otherwise, input the number of products.**
5. **If the number of products is less than or equal to zero, display an invalid product count message.**
6. **For each product:**
    - **Input the product name.**
    - **Input the price per unit.**
    - **Input the quantity.**
    - **Validate the price and quantity.**
    - **Calculate the product cost using:**

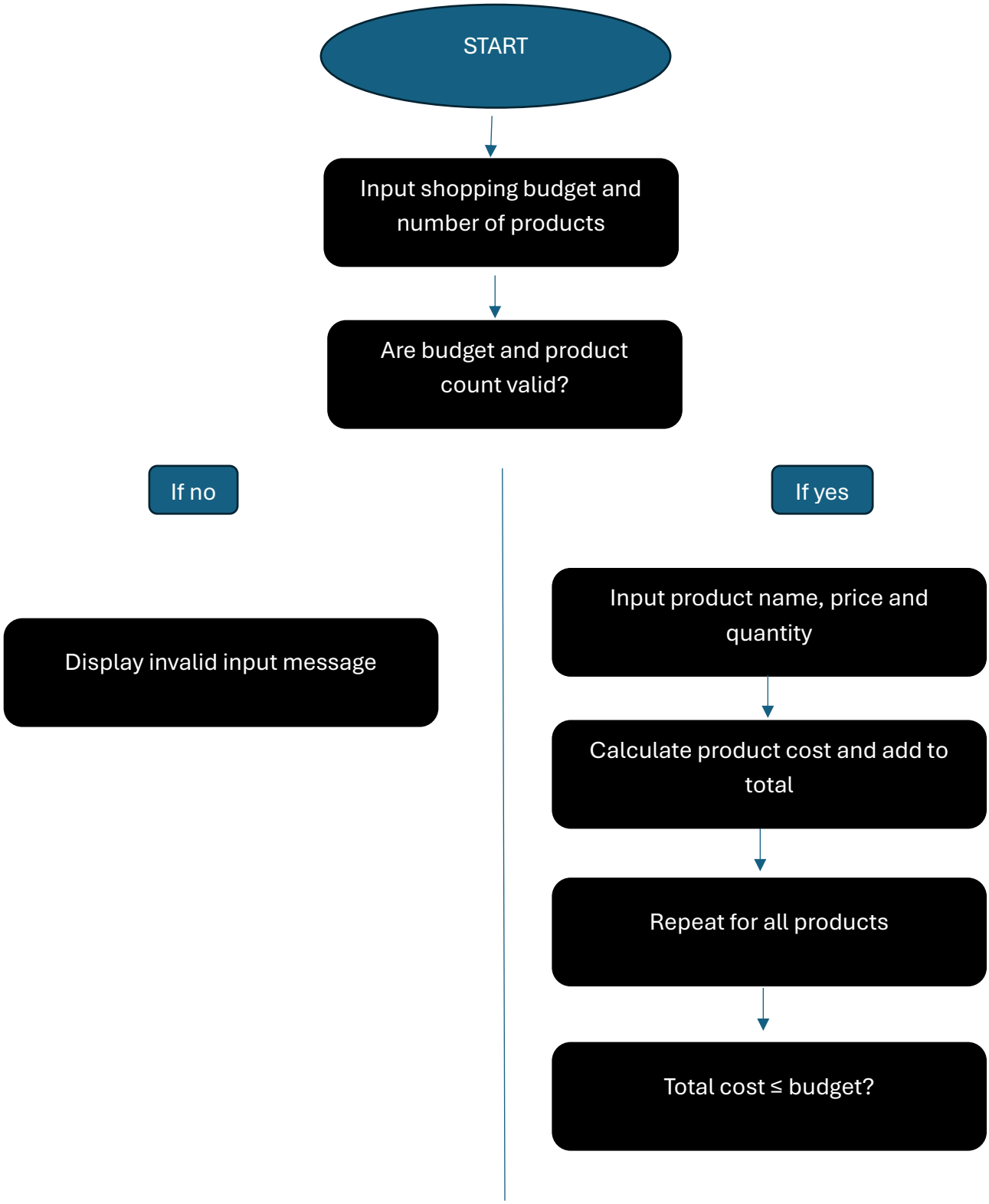
Product Cost = Price × Quantity

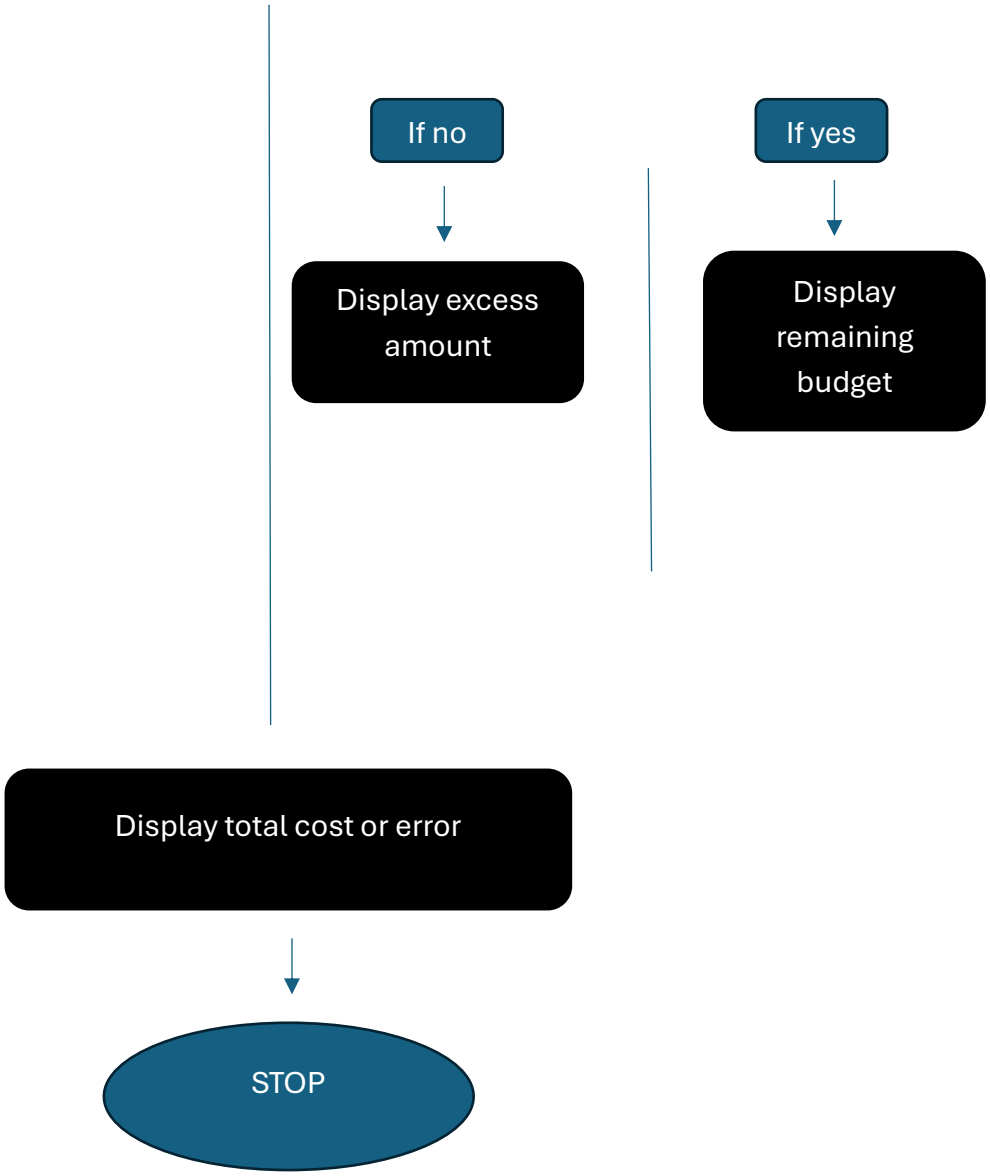
    - **Add the product cost to the total shopping cost.**
7. **If all product details are valid, display the total shopping cost.**
8. **Compare the total shopping cost with the budget.**
9. **If the total cost is less than or equal to the budget, display the remaining budget.**

10. Otherwise, display the amount by which the budget is exceeded.

11. Stop the program.

5.Flow chart





6. Pseudocode

```
BEGIN

INPUT budget
INPUT number_of_products
IF budget <= 0 THEN
   PRINT "Invalid budget"

ELSE IF number_of_products <= 0 THEN
```

```
    PRINT "Invalid number of products"


ELSE
    SET total_cost = 0
    SET valid = TRUE
FOR each product from 1 to number_of_products DO


        INPUT product_name
        INPUT price
        INPUT quantity


        IF price <= 0 OR quantity <= 0 THEN
            PRINT "Invalid product details"
            SET valid = FALSE
            STOP LOOP


        ELSE
            product_cost = price * quantity
            total_cost = total_cost + product_cost
            PRINT product_name
            PRINT product_cost
        END IF
```

```
  END FOR

  IF valid = TRUE THEN
    PRINT total_cost

    IF total_cost <= budget THEN
      remaining = budget - total_cost
      PRINT "Within budget"
      PRINT remaining
    ELSE
      excess = total_cost - budget
      PRINT "Budget exceeded"
      PRINT excess
    END IF
  END IF

END IF
END
```

7.source code

Multi-Product Shopping Budget Constraint.py

PYTHON-ACTIVITY-BASED-LEARNING > Multi-Product Shopping Budget Constraint.py > ...

```python

budget = float(input("Enter shopping budget: Rs. "))
n = int(input("Enter number of products: "))

if budget <= 0:
    print("Invalid budget")
elif n <= 0:
    print("Invalid number of products")
else:
    total_cost = 0
    valid = True

    for i in range(n):
        name = input("Enter product name: ")
        price = float(input("Enter price per unit: Rs. "))
        quantity = int(input("Enter quantity: "))

        if price <= 0 or quantity <= 0:
            print("Invalid product details")
            valid = False
            break

        product_cost = price * quantity
        total_cost += product_cost

        print("Product:", name)
        print("Product cost: Rs.", round(product_cost, 2))
    if valid:
        print("Total shopping cost: Rs.", round(total_cost, 2))

        if total_cost <= budget:
            remaining = budget - total_cost
            print("Shopping is within budget")
            print("Remaining budget: Rs.", round(remaining, 2))
        else:
            excess = total_cost - budget
            print("Budget exceeded")
            print("Excess amount: Rs.", round(excess, 2))

```

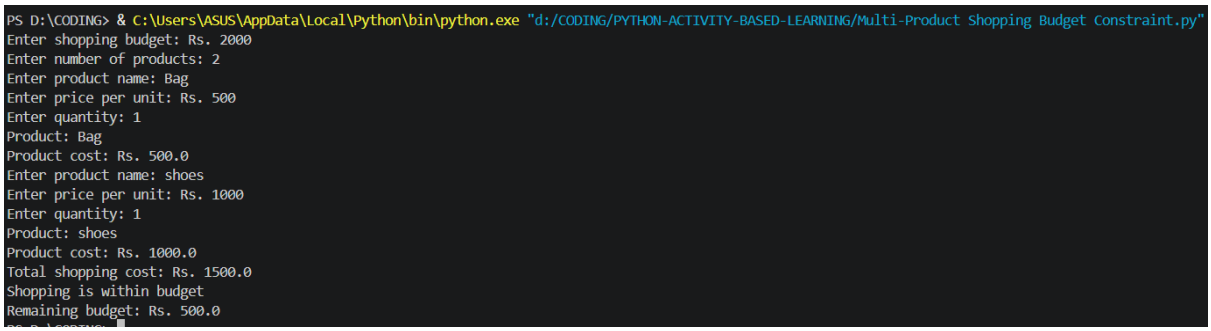
8. Test Cases

Test Case	Input	Expected Output
1. Within budget	Budget: ₹2,000; 2 products: Bag ₹500 × 1, Shoes ₹1,000 × 1	Total cost: ₹1,500; Shopping is within budget; Remaining budget: ₹500
2. Budget exceeded	Budget: ₹1,000; 2 products: Watch ₹800 × 1, Headphones ₹500 × 1	Total cost: ₹1,300; Budget exceeded; Excess amount: ₹300

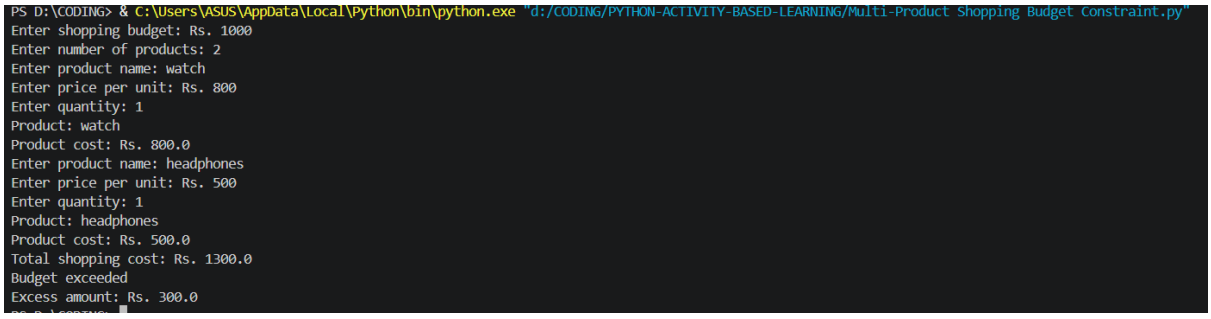
3. Exact budget	Budget: ₹1,000; 1 product: Keyboard ₹1,000 × 1	Total cost: ₹1,000; Shopping is within budget; Remaining budget: ₹0
4. Invalid budget	Budget: ₹0	Invalid budget
5. Invalid product details	Budget: ₹2,000; 1 product: Bag ₹500 × 0	Invalid product details

9. Output Screenshots

Screenshot 1: Program output showing a shopping purchase within the available budget.



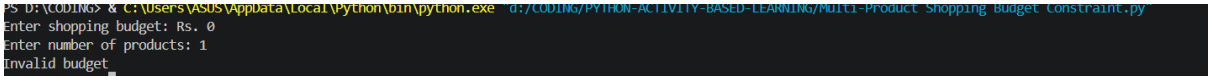
Screenshot 2: Program output showing the shopping cost exceeding the available budget.



Screenshot 3: Program output showing a purchase that exactly matches the budget.



Screenshot 4: Program output showing an invalid shopping budget.



10. Conclusion

The Multi-Product Shopping Budget Constraint program was developed successfully using Python. It accepts the customer's budget and product details, calculates the cost of each product, and determines the total shopping expense. The program compares the total cost with the available budget and displays the remaining budget or excess amount. It also validates budget values, product prices, and quantities to prevent invalid calculations. This project demonstrates the practical application of variables, loops, conditional statements, arithmetic operations, and input validation in Python. It helps users manage their shopping expenses effectively.